Design and Analysis of Algorithms — Lab 08

Dynamic Programming Applications

Submitted By: Vaibhav Maheswari | **Roll No:** B125136 | **Class:** CSE-B-2 | **Date:** _____

Q1. Minimum Coin Change

Problem Statement

Given an integer array of coin denominations C = {c_1, c_2, ..., c_n} representing coins of different values, and an integer target amount V, find the minimum number of coins needed to make up that amount. Infinite supply of each denomination is assumed. If the amount cannot be made, return −1.

Complexity Analysis

- **Approach:** Bottom-up DP. dp[i] = min coins to make amount *i*. Recurrence: `dp[i] = 1 + min(dp[i − c])` over all coins c ≤ i.
- **Time:** O(n · V) — for every amount 1..V, all n coins checked.
- **Space:** O(V) — dp[] and last[] arrays.

C Implementation

```c
#include <stdio.h>
#include <stdlib.h>

#define INF 1000000000

int minCoins(int coins[], int n, int V) {
    int i, j;
    int *dp = (int *)malloc((V + 1) * sizeof(int));
    int *last = (int *)malloc((V + 1) * sizeof(int));
    if (dp == NULL || last == NULL) { printf("Memory allocation failed!\n"); return -1; }

    dp[0] = 0; last[0] = -1;
    for (i = 1; i <= V; i++) { dp[i] = INF; last[i] = -1; }

    for (i = 1; i <= V; i++) {
        for (j = 0; j < n; j++) {
            int c = coins[j];
            if (c <= i && dp[i - c] != INF) {
                if (1 + dp[i - c] < dp[i]) {
                    dp[i] = 1 + dp[i - c];
                    last[i] = c;
                }
            }
        }
    }

    printf("\n--- DP Table (Amount -> Min Coins) ---\n");
    printf("Amount: ");
    for (i = 0; i <= V; i++) printf("%4d ", i);
    printf("\nCoins: ");
    for (i = 0; i <= V; i++) {
        if (dp[i] == INF) printf(" INF ");
        else printf("%4d ", dp[i]);
    }
    printf("\n");

    if (dp[V] != INF) {
        printf("\nCoins used to form %d: ", V);
        int temp = V;
        while (temp > 0) { printf("%d ", last[temp]); temp -= last[temp]; }
        printf("\n");
    }

    int ans = dp[V];
    free(dp); free(last);
```

```
    return (ans >= INF) ? -1 : ans;
}

int main() {
    int n, V, i;
    printf("Minimum Coin Change Problem (Lab 08)\n");
    printf("Enter number of coin denominations: ");
    if (scanf("%d", &n) != 1 || n <= 0) { printf("Invalid.\n"); return 1; }

    int *coins = (int *)malloc(n * sizeof(int));
    if (!coins) { printf("Alloc failed!\n"); return 1; }

    printf("Enter the %d coin denominations: ", n);
    for (i = 0; i < n; i++) scanf("%d", &coins[i]);

    printf("Enter target amount V: ");
    if (scanf("%d", &V) != 1 || V < 0) { printf("Invalid.\n"); free(coins); return 1; }

    int ans = minCoins(coins, n, V);
}   if (ans == -1) printf("\nAmount %d cannot be made. Answer = -1\n", V);
    else           printf("\nMinimum coins needed = %d\n", ans);

    free(coins);
    return 0;
```

Sample Input / Output

```
Enter number of coin denominations: 3
Enter the 3 coin denominations: 1 5 6
Enter target amount V: 11

--- DP Table (Amount -> Min Coins) ---
Amount:   0   1   2   3   4   5   6   7   8   9  10  11
Coins:    0   1   2   3   4   1   1   2   2   3   2   2
Coins used to form 11: 6 5

Minimum coins needed = 2
```

Q2. Coin Change — Total Number of Ways

Problem Statement

> Given an array of distinct positive integers $C = \{c_1, \ldots, c_n\}$ and target V, find the total number of distinct combinations of coins that sum to V. Infinite supply assumed. Order does not matter ($1+2 \equiv 2+1$).

Complexity Analysis

- **Approach:** Bottom-up DP. `dp[j] += dp[j − c]`. Outer loop over coins ensures combinations, not permutations.
- **Time:** $O(n \cdot V)$
- **Space:** $O(V)$

C Implementation

```c
#include <stdio.h>
#include <stdlib.h>

unsigned long long countWays(int coins[], int n, int V) {
    int i, j;
    unsigned long long *dp = (unsigned long long *)calloc(V + 1, sizeof(unsigned long long));
    if (!dp) { printf("Memory allocation failed!\n"); return 0; }

    dp[0] = 1; /* one way to make 0 (pick nothing) */

    /* Outer loop over coins -> combinations not permutations */
    for (i = 0; i < n; i++) {
    int c = coins[i];
    for (j = c; j <= V; j++) {
            dp[j] += dp[j - c];
        }
    }

    printf("\n--- DP Table (Amount -> Distinct Ways) ---\n");
    for (j = 0; j <= V; j++)
        printf("Amount %2d : %llu ways\n", j, dp[j]);
}

int main() {
    unsigned long long total = dp[V];
    free(dp);
    return total;


    int n, V, i;
    printf("Coin Change: Total Number of Ways\n");
    printf("Enter number of distinct coin denominations: ");
    if (scanf("%d", &n) != 1 || n <= 0) { printf("Invalid.\n"); return 1; }

    int *coins = (int *)malloc(n * sizeof(int));
    if (!coins) { printf("Alloc failed!\n"); return 1; }

    printf("Enter the %d distinct coin denominations: ", n);
    for (i = 0; i < n; i++) scanf("%d", &coins[i]);

    printf("Enter target amount V: ");
    if (scanf("%d", &V) != 1 || V < 0) { printf("Invalid.\n"); free(coins); return 1; }
}
    unsigned long long result = countWays(coins, n, V);
    printf("\nTotal distinct combinations to form %d: %llu\n", V, result);

    free(coins);
    return 0;
```

Q3. Longest Common Subsequence with Reconstruction

Problem Statement

Given two sequences X = ⟨x_1, ..., x_m⟩ and Y = ⟨y_1, ..., y_n⟩, compute the length of their longest common subsequence and reconstruct the actual subsequence string.

Complexity Analysis

- **Approach:** Fill 2D DP table of size (m+1)×(n+1), then backtrack from dp[m][n] to reconstruct LCS.
- **Time:** O(m · n) for table + O(m + n) backtrack.
- **Space:** O(m · n) for 2D DP.

C Implementation

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LEN 1024

int max(int a, int b) { return (a > b) ? a : b; }

void findLCS(char X[], char Y[]) {
    int m = strlen(X), n = strlen(Y), i, j;

    int **dp = (int **)malloc((m + 1) * sizeof(int *));
    for (i = 0; i <= m; i++) dp[i] = (int *)malloc((n + 1) * sizeof(int));

    for (i = 0; i <= m; i++) {
        for (j = 0; j <= n; j++) {
            if (i == 0 || j == 0)                dp[i][j] = 0;
            else if (X[i-1] == Y[j-1])           dp[i][j] = 1 + dp[i-1][j-1];
            else                                   dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }

    printf("\n--- 2D DP Table (%d x %d) ---\n", m + 1, n + 1);
    printf("      ");
    for (j = 0; j < n; j++) printf(" %c", Y[j]);
    printf("\n");
    for (i = 0; i <= m; i++) {
        if (i == 0) printf("    ");
        else printf(" %c ", X[i-1]);
        for (j = 0; j <= n; j++) printf("%3d", dp[i][j]);
        printf("\n");
    }

    int lcs_len = dp[m][n];
    printf("\nLength of LCS: %d\n", lcs_len);

    char *lcs = (char *)malloc((lcs_len + 1) * sizeof(char));
    lcs[lcs_len] = '\0';
    i = m; j = n;
    int idx = lcs_len - 1;

    while (i > 0 && j > 0) {
        if (X[i-1] == Y[j-1]) { lcs[idx--] = X[i-1]; i--; j--; }
        else if (dp[i-1][j] >= dp[i][j-1]) i--;
        else j--;
    }

    printf("Reconstructed LCS: \"%s\"\n", lcs);

    free(lcs);
    for (i = 0; i <= m; i++) free(dp[i]);
    free(dp);
}
int main() {
```

```
    char X[MAX_LEN], Y[MAX_LEN];
    printf("Longest Common Subsequence (LCS) with Reconstruction\n");
    printf("Enter first sequence (X): ");
    if (scanf("%s", X) != 1) return 1;
    printf("Enter second sequence (Y): ");
    if (scanf("%s", Y) != 1) return 1;
    findLCS(X, Y);
    return 0;
}
```

Sample Input / Output

```
Enter first sequence (X): AGGTAB Enter
second sequence (Y): GXTXAYB

--- 2D DP Table (7 x 8) ---
          G X T X A Y B
0 0 0 0 0 0 0 0
A 0 0 0 0 0 1 1 1
G 0 1 1 1 1 1 1 1
G 0 1 1 1 1 1 1 1
T 0 1 1 2 2 2 2 2
A 0 1 1 2 2 3 3 3
B 0 1 1 2 2 3 3 4

Length of LCS: 4
Reconstructed LCS: "GTAB"
```

Q4. Longest Increasing Subsequence

Problem Statement

> Given an integer array A = [a_0, a_1, ..., a_{n-1}], find the length of the longest subsequence such that all elements are strictly increasing, and reconstruct the subsequence.

Complexity Analysis

- **Approach:** `dp[i]` = LIS ending at i. `dp[i] = 1 + max(dp[j])` for all j < i with A[j] < A[i]. Parent array tracks the path.
- **Time:** `O(n²)` — nested loops.
- **Space:** `O(n)` — `dp[]` and `parent[]`.

C Implementation

```c
#include <stdio.h>
#include <stdlib.h>

void computeLIS(int A[], int n) {
    if (n <= 0) return;
    int i, j;

    int *dp = (int *)malloc(n * sizeof(int));
    int *parent = (int *)malloc(n * sizeof(int));
    if (!dp || !parent) { printf("Alloc failed!\n"); return; }

    for (i = 0; i < n; i++) { dp[i] = 1; parent[i] = -1; }

    for (i = 1; i < n; i++) {
        for (j = 0; j < i; j++) {
            if (A[j] < A[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
                parent[i] = j;
            }
        }
    }

    int maxLen = dp[0], maxIdx = 0;
    for (i = 1; i < n; i++)
        if (dp[i] > maxLen) { maxLen = dp[i]; maxIdx = i; }

    printf("\n--- DP Table ---\n");
    printf("Index: ");
    for (i = 0; i < n; i++) printf("%4d ", i);
    printf("\nArray: ");
    for (i = 0; i < n; i++) printf("%4d ", A[i]);
    printf("\nDP:    ");
    for (i = 0; i < n; i++) printf("%4d ", dp[i]);
    printf("\n\nLength of LIS: %d\n", maxLen);

    int *seq = (int *)malloc(maxLen * sizeof(int));
    int cur = maxIdx, k = maxLen - 1;
    while (cur != -1) { seq[k--] = A[cur]; cur = parent[cur]; }

    printf("Reconstructed LIS: [ ");
    for (i = 0; i < maxLen; i++) printf("%d ", seq[i]);
    printf("]\n");

    free(seq); free(dp); free(parent);
}

int main() {
    int n, i;
    printf("Longest Increasing Subsequence (LIS)\n");
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) { printf("Invalid.\n"); return 1; }

    int *A = (int *)malloc(n * sizeof(int));
    if (!A) { printf("Alloc failed!\n"); return 1; }
```

```
    printf("Enter %d integers: ", n);
    for (i = 0; i < n; i++) scanf("%d", &A[i]);

    computeLIS(A, n);
    free(A);
    return 0;
}
```

Sample Input / Output

```
Enter number of elements: 8
Enter 8 integers: 10 22 9 33 21 50 41 60

--- DP Table ---
Index:   0   1   2   3   4   5   6   7
Array:  10  22   9  33  21  50  41  60
DP:      1   2   1   3   2   4   3   5

Length of LIS: 5
Reconstructed LIS: [ 10 22 33 50 60 ]
```

Q5. Maximum Sum Increasing Subsequence (MSIS)

Problem Statement

> Given an array of n positive integers A = [a_0, ..., a_{n-1}], find the maximum possible sum of a strictly increasing subsequence, and reconstruct it.

Complexity Analysis

- **Approach:** `msis[i] = max(msis[j] + A[i])` for j < i with A[j] < A[i], else A[i]. Parent array for reconstruction.
- **Time:** $O(n^2)$
- **Space:** $O(n)$

C Implementation

```c
#include <stdio.h>
#include <stdlib.h>

void computeMSIS(int A[], int n) {
    if (n <= 0) return;

    int *msis = (int *)malloc(n * sizeof(int));
    int *parent = (int *)malloc(n * sizeof(int));
    if (!msis || !parent) { printf("Alloc failed!\n"); return; }

    for (int i = 0; i < n; i++) { msis[i] = A[i]; parent[i] = -1; }

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (A[j] < A[i] && msis[j] + A[i] > msis[i]) {
                msis[i] = msis[j] + A[i];
                parent[i] = j;
            }
        }
    }

    int max_sum = msis[0], max_index = 0;
    for (int i = 1; i < n; i++)
        if (msis[i] > max_sum) { max_sum = msis[i]; max_index = i; }

    printf("\n--- DP Table ---\n");
    printf("Index: ");
    for (int i = 0; i < n; i++) printf("%4d ", i);
    printf("\nArray: ");
    for (int i = 0; i < n; i++) printf("%4d ", A[i]);
    printf("\nMSIS: ");
    for (int i = 0; i < n; i++) printf("%4d ", msis[i]);
    printf("\n\nMaximum Sum of Increasing Subsequence: %d\n", max_sum);

    int *seq = (int *)malloc(n * sizeof(int));
    int count = 0, curr = max_index;
    while (curr != -1) { seq[count++] = A[curr]; curr = parent[curr]; }

    printf("Subsequence achieving maximum sum: [ ");
    for (int i = count - 1; i >= 0; i--) printf("%d ", seq[i]);
    printf("]\n");

    free(seq); free(msis); free(parent);
}

int main() {

    int n;
    printf("Maximum Sum Increasing Subsequence (MSIS)\n");
    printf("Enter number of positive integers: ");
    if (scanf("%d", &n) != 1 || n <= 0) { printf("Invalid.\n"); return 1; }

    int *A = (int *)malloc(n * sizeof(int));
    if (!A) { printf("Alloc failed!\n"); return 1; }

    printf("Enter %d positive integers: ", n);
```

```
    for (int i = 0; i < n; i++) scanf("%d", &A[i]);

    computeMSIS(A, n);
    free(A);
    return 0;
}
```

Sample Input / Output

```
Enter number of positive integers: 7
Enter 7 positive integers: 1 101 2 3 100 4 5

--- DP Table ---
Index:   0   1   2   3   4   5   6
Array:       1 101   2   3 100   4   5
MSIS:        1 101   3   6 106

Maximum Sum of Increasing Subsequence: 106
Subsequence achieving maximum sum: [ 1 2 3 100 ]
```

Q6. Edit Distance with Traceback

Problem Statement

> Given strings A (length m) and B (length n), compute the minimum number of operations (insert, delete, substitute) required to transform A into B, and print the step-by-step traceback.

Complexity Analysis

- **Approach:** `dp[i][j] = dp[i−1][j−1]` if match, else `1 + min(delete, insert, substitute)`. Traceback reconstructs the operations.
- **Time:** $O(m \cdot n)$
- **Space:** $O(m \cdot n)$

C Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LEN 1024

int min3(int a, int b, int c) {
    int m = a;
    if (b < m) m = b; if (c <
    m) m = c; return m;

}

typedef struct { char action[64]; } EditStep; void
computeEditDistance(char A[], char B[]) {

    int m = strlen(A), n = strlen(B);

    int **dp = (int **)malloc((m + 1) * sizeof(int *));
    for (int i = 0; i <= m; i++) dp[i] = (int *)malloc((n + 1) * sizeof(int));

    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (A[i-1] == B[j-1]) dp[i][j] = dp[i-1][j-1];
            else dp[i][j] = 1 + min3(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
        }
    }

    printf("\n--- 2D DP Table ---\n                    #");
    for (int j = 0; j < n; j++) printf(" %c", B[j]);
    printf("\n");
    for (int i = 0; i <= m; i++) {
    if (i == 0) printf(" # ");
    else printf(" %c ", A[i-1]);
    for (int j = 0; j <= n; j++) printf("%3d", dp[i][j]);
    printf("\n");
    }

    printf("\nMin Edit Distance: %d\n", dp[m][n]);

    EditStep *steps = (EditStep *)malloc((m + n + 1) * sizeof(EditStep));
B[j-1]);
    int step_count = 0, i = m, j = n;

    while (i > 0 || j > 0) {
        if (i > 0 && j > 0 && A[i-1] == B[j-1] && dp[i][j] == dp[i-1][j-1]) {
            snprintf(steps[step_count++].action, 64, "Keep '%c' (Match)", A[i-1]);
            i--; j--;
        } else if (i > 0 && j > 0 && dp[i][j] == dp[i-1][j-1] + 1) {
            snprintf(steps[step_count++].action, 64, "Substitute '%c' with '%c'", A[i-1],

            i--; j--;
```

```
            } else if (i > 0 && dp[i][j] == dp[i-1][j] + 1) {
                snprintf(steps[step_count++].action, 64, "Delete '%c' from A", A[i-1]);
                i--;
            } else if (j > 0 && dp[i][j] == dp[i][j-1] + 1) {
                snprintf(steps[step_count++].action, 64, "Insert '%c' into A", B[j-1]);
                j--;
            }
        }

        printf("\n--- Step-by-Step Traceback ---\n");
        int op_num = 1;
        for (int k = step_count - 1; k >= 0; k--)
            printf("Step %2d: %s\n", op_num++, steps[k].action);

        free(steps);
}       for (int r = 0; r <= m; r++) free(dp[r]);
        free(dp);
int main() {


        char A[MAX_LEN], B[MAX_LEN];
        printf("Edit Distance with Traceback\n"); printf("Enter
        source string A: ");
        if (scanf("%s", A) != 1) return 1; printf("Enter target
        string B: ");
        if (scanf("%s", B) != 1) return 1;
        computeEditDistance(A, B);
        return 0;
}
```

Sample Input / Output

```
Enter source string A: kitten Enter target
string B: sitting

Min Edit Distance: 3

--- Step-by-Step Traceback --- Step 1: Keep
'k' (Match)
Step 2: Substitute 'i' with 'i' Step 3:
Substitute 't' with 't' Step 4: Substitute 't'
with 't' Step 5: Insert 'i' into A
Step 6: Keep 'e' (Match)
Step 7: Substitute 'n' with 'n'
```

Q7. Rod Cutting with Reconstruction

Problem Statement

> Given rod length n and price array P = [p_1, ..., p_n], determine (i) the maximum revenue obtainable by cutting the rod and (ii) the exact piece lengths giving the optimal decomposition.

Complexity Analysis

- **Approach:** `revenue[j] = max(price[i−1] + revenue[j−i])`. `first_cut[]` enables reconstruction.
- **Time:** $O(n^2)$ — inner loop tries cuts 1..j.
- **Space:** $O(n)$ — `revenue[]`, `first_cut[]`.

C Implementation

```c
#include <stdio.h>
#include <stdlib.h>

void rodCutting(int price[], int n) {
    int *revenue = (int *)malloc((n + 1) * sizeof(int));
    int *first_cut = (int *)malloc((n + 1) * sizeof(int));
    if (!revenue || !first_cut) { printf("Alloc failed!\n"); return; }

    revenue[0] = 0; first_cut[0] = 0;

    for (int j = 1; j <= n; j++) {
        int max_val = -1, best_cut = -1;
        for (int i = 1; i <= j; i++) {
            int curr = price[i-1] + revenue[j-i];
            if (curr > max_val) { max_val = curr; best_cut = i; }
        }
        revenue[j] = max_val;
        first_cut[j] = best_cut;
    }

    printf("\n--- DP Table ---\n");
    printf("Length:  ");
    for (int j = 0; j <= n; j++) printf("%4d ", j);
    printf("\nRevenue: ");
    for (int j = 0; j <= n; j++) printf("%4d ", revenue[j]);
    printf("\nFirst Cut: ");
    for (int j = 0; j <= n; j++) printf("%4d ", first_cut[j]);

    printf("\n\n(i) Maximum Revenue Obtainable: %d\n", revenue[n]);

    printf("(ii) Optimal Piece Lengths: [ ");
    int temp = n, piece_count = 0;
    while (temp > 0) {
        int cut = first_cut[temp];
        printf("%d ", cut);
        temp -= cut;
        piece_count++;
    }
    printf("]\n  Total pieces: %d (sum = %d)\n", piece_count, n);

    free(revenue); free(first_cut);
}

int main() {
    int n;
    printf("Rod Cutting with Piece Length Reconstruction\n");
    printf("Enter total length of rod (n): ");
    if (scanf("%d", &n) != 1 || n <= 0) { printf("Invalid.\n"); return 1; }

    int *price = (int *)malloc(n * sizeof(int));
    if (!price) { printf("Alloc failed!\n"); return 1; }

    printf("Enter prices for lengths 1 to %d:\n", n);
    for (int i = 0; i < n; i++) {
        printf("  Price for length %2d: ", i + 1);
```

```
            scanf("%d", &price[i]);
        }

        rodCutting(price, n);
        free(price);
        return 0;
}
```

Sample Input / Output

```
Enter total length of rod (n): 8
Price for length 1: 1
Price for length 2: 5
Price for length 3: 8
Price for length 4: 9
Price for length 5: 10
Price for length 6: 17
Price for length 7: 17
Price for length 8: 20

(i) Maximum Revenue Obtainable: 22
(ii) Optimal Piece Lengths: [ 6 2 ]
Total pieces: 2 (sum = 8)
```

Q8. Optimal Binary Search Tree (OBST)

Problem Statement

> Given n sorted keys K = ⟨k_1, ..., k_n⟩ with search probabilities $p_1..p_n$ and n+1 dummy keys $d_0..d_n$ with probabilities $q_0..q_n$, find the minimum expected search cost of a BST and print the optimal tree structure.

Complexity Analysis

- **Approach:** 3 nested loops (length l, start i, root r). Tables `e[i][j]`, `w[i][j]`, `root[i][j]`. Recurrence `e[i][j] = min(e[i][r−1] + e[r+1][j] + w[i][j])`.
- **Time:** $O(n^3)$
- **Space:** $O(n^2)$

C Implementation

```c
#include <stdio.h>
#include <stdlib.h>
#include <float.h>

#define MAX_KEYS 100

void printOptimalBST(int root[][MAX_KEYS], int i, int j, int parent, int is_left) {
    if (i > j) {
        if (parent == 0) printf("d%d is the root (empty tree)\n", j);
        else if (is_left) printf("d%d is left child of k%d\n", j, parent);
        else printf("d%d is right child of k%d\n", j, parent);
        return;
    }
    int r = root[i][j];
    if (parent == 0) printf("k%d is the root of the tree\n", r);
    else if (is_left) printf("k%d is left child of k%d\n", r, parent);
    else printf("k%d is right child of k%d\n", r, parent);
    printOptimalBST(root, i, r - 1, r, 1);
    printOptimalBST(root, r + 1, j, r, 0);
}

void optimalBST(double p[], double q[], int n) {
    double e[MAX_KEYS][MAX_KEYS];
    double w[MAX_KEYS][MAX_KEYS];
    int root[MAX_KEYS][MAX_KEYS];

    for (int i = 1; i <= n + 1; i++) {
        e[i][i-1] = q[i-1]; w[i][i-1]
        = q[i-1];
    }

    for (int l = 1; l <= n; l++) {
        for (int i = 1; i <= n - l + 1; i++) {
            int j = i + l - 1;
            e[i][j] = DBL_MAX;
            w[i][j] = w[i][j-1] + p[j] + q[j]; for (int r = i; r <=
            j; r++) {
                double cost = e[i][r-1] + e[r+1][j] + w[i][j];
                if (cost < e[i][j]) { e[i][j] = cost; root[i][j] = r; }
            }
        }
    }

    printf("\n--- Expected Cost Table e[i][j] ---\n");
    printf("  j=");
    for (int j = 0; j <= n; j++) printf("%8d", j);
    printf("\n");
    for (int i = 1; i <= n + 1; i++) {
        printf("i=%2d:", i);
        for (int j = 0; j <= n; j++) {
            if (j >= i - 1) printf("%8.4f", e[i][j]);
            else printf("        ");
        }
        printf("\n");
```

```
    }

    printf("\n--- Root Table root[i][j] ---\n");
    printf("      j=");
    for (int j = 1; j <= n; j++) printf("%5d", j);
    printf("\n");
    for (int i = 1; i <= n; i++) {
        printf("i=%2d:", i);
        for (int j = 1; j <= n; j++) {
            if (j >= i) printf("%5d", root[i][j]);
            else        printf("     ");
        }
        printf("\n");
    }

    printf("\nMinimum Expected Search Cost: %.4f\n", e[1][n]);
}
    printf("\n--- Optimal BST Structure ---\n");
int main() {    printOptimalBST(root, 1, n, 0, 0);


    int n;
    printf("Optimal Binary Search Trees (OBST)\n");
    printf("Enter number of keys (n): ");
    if (scanf("%d", &n) != 1 || n <= 0 || n >= MAX_KEYS - 2) { printf("Invalid.\n");
return 1; }

    double p[MAX_KEYS], q[MAX_KEYS];

    printf("Enter p[1..%d] (successful search probabilities):\n", n);
    for (int i = 1; i <= n; i++) { printf(" p[%d]: ", i); scanf("%lf", &p[i]); }

    printf("Enter q[0..%d] (unsuccessful search probabilities):\n", n);
    for (int i = 0; i <= n; i++) { printf(" q[%d]: ", i); scanf("%lf", &q[i]); }

    optimalBST(p, q, n);
}   return 0;
```

Sample Input / Output

```
Enter number of keys (n): 3
p[1]: 0.3       p[2]: 0.2       p[3]: 0.2
q[0]: 0.05 q[1]: 0.1 q[2]: 0.05 q[3]: 0.1 Minimum Expected
Search Cost: 1.9000

--- Optimal BST Structure ---
k2 is the root of the tree
k1 is left child of k2
d0 is left child of k1
d1 is right child of k1
k3 is right child of k2
d2 is left child of k3
d3 is right child of k3
```

Q9. Collatz Conjecture (3n + 1) — Open Problem

Problem Statement

> The Collatz Conjecture defines: T(n) = n/2 if n is even; T(n) = 3n + 1 if n is odd. The sequence stops when n = 1. Write a modular C program to analyse the trajectory of a starting value n ≥ 1 and across an interval [a, b]. Handles integer overflow using 64-bit integers and dynamic memory for trajectory storage.

Complexity Analysis

- **Single n:** Time O(L(n)); Space O(L(n)) on heap, where L(n) = trajectory length.
- **Interval [a,b]:** Time O(Σ L(k)) for k ∈ [a,b]; Space O(max L(k)) (buffer freed per number).
- **Note:** L(n) has no proven closed form — Collatz remains an open problem.

C Implementation

```
#include <stdio.h>
#include <stdlib.h>

/* Next Collatz number */
unsigned long long getNextCollatz(unsigned long long n) {
    if (n % 2 == 0) return n / 2;
    else return (3ULL * n) + 1ULL;
}

/* Build trajectory using dynamic memory (malloc + realloc) */
unsigned long long* computeTrajectory(unsigned long long n, int *out_steps, unsigned long long *out_peak) {

    int capacity = 64, count = 0;
    unsigned long long *trajectory = (unsigned long long *)malloc(capacity *
sizeof(unsigned long long));
    if (!trajectory) { printf("Alloc failed!\n"); *out_steps = 0; *out_peak = n; return
NULL; }

    unsigned long long curr = n, peak = n;
    trajectory[count++] = curr;

    while (curr != 1ULL) {
        curr = getNextCollatz(curr);
        if (curr > peak) peak = curr;

        if (count >= capacity) {
            capacity *= 2;
            unsigned long long *temp = (unsigned long long *)realloc(trajectory, capacity
* sizeof(unsigned long long));
            if (!temp) { printf("Realloc failed!\n"); free(trajectory); *out_steps = 0;
*out_peak = peak; return NULL; }
            trajectory = temp;
        }
        trajectory[count++] = curr;
    }

    *out_steps = count;
    *out_peak = peak;
}   return trajectory;

/* Analyze a single starting value */
void analyzeSingleValue(unsigned long long n) {

    if (n < 1ULL) { printf("Starting number must be >= 1.\n"); return; }

    printf("\n--- Collatz Analysis for n = %llu ---\n", n);
    int steps = 0;
    unsigned long long peak = 0;
    unsigned long long *trajectory = computeTrajectory(n, &steps, &peak); if (!trajectory)
    return;

    printf("Trajectory points (incl. start): %d\n", steps);
    printf("Step transitions to reach 1: %d\n", steps - 1);
    printf("Peak value reached: %llu\n\n", peak);
```

```
    printf("Trajectory Path:\n");
    for (int i = 0; i < steps; i++) {
        printf("%llu", trajectory[i]);
        if (i < steps - 1) printf(" -> ");
        if ((i + 1) % 8 == 0 && i < steps - 1) printf("\n    ");
    }
    printf("\n");
    free(trajectory);
}

/* Analyze interval [a, b] */
void analyzeInterval(unsigned long long a, unsigned long long b) {
    if (a > b || a < 1ULL) { printf("Invalid interval.\n"); return; }

    printf("\n--- Collatz Analysis for [%llu, %llu] ---\n", a, b);

    int max_length = 0;

    unsigned long long num_max_len = a;

    unsigned long long global_peak = 0, num_peak = a;

    printf("%-12s %-15s %-20s\n", "Number (n)", "Steps to 1", "Peak Value");

    printf("-------------------------------------------------------\n");

    for (unsigned long long k = a; k <= b; k++) {
        int steps = 0;
        unsigned long long peak = 0;
        unsigned long long *trajectory = computeTrajectory(k, &steps, &peak);
        if (trajectory) {
        int transitions = steps - 1;
        if ((b - a + 1) <= 50)
            printf("%-12llu %-15d %-20llu\n", k, transitions, peak);
        if (transitions > max_length) { max_length = transitions; num_max_len = k; }
        if (peak > global_peak)       { global_peak = peak;       num_peak = k; }
        free(trajectory);
    }
    }
}
int main() {
    printf("-------------------------------------------------------\n");
    printf("Max Trajectory Length: %d (n = %llu)\n", max_length, num_max_len);
    printf("Highest Peak Value:    %llu (n = %llu)\n", global_peak, num_peak);
    printf("-------------------------------------------------------\n");


    int    choice;
    while (1) {
        printf("\n=== Collatz Conjecture (3n + 1) Modular System ===\n");
        printf("1. Analyze single starting value n\n");
        printf("2. Analyze an interval [a, b]\n");
        printf("3. Exit\n");
        printf("Enter your choice (1-3): ");
        if(scanf("%d", &choice) != 1) break;

        if(choice == 1) {
            unsigned long long n;
            printf("Enter starting integer n (>= 1): ");
            if(scanf("%llu", &n) == 1) analyzeSingleValue(n);
        }elseif (choice == 2) {
            unsigned long long a, b;
            printf("Enter lower bound a: "); scanf("%llu", &a); printf("Enter
            upper bound b: "); scanf("%llu", &b); analyzeInterval(a, b);

        }elseif (choice == 3) {
            printf("Exiting Collatz program. Goodbye!\n");
            break;
        } else {
            printf("Invalid choice! Choose 1, 2, or 3.\n");
        }
    }
    return 0;
}
```